

Project 5A: BYOW Design

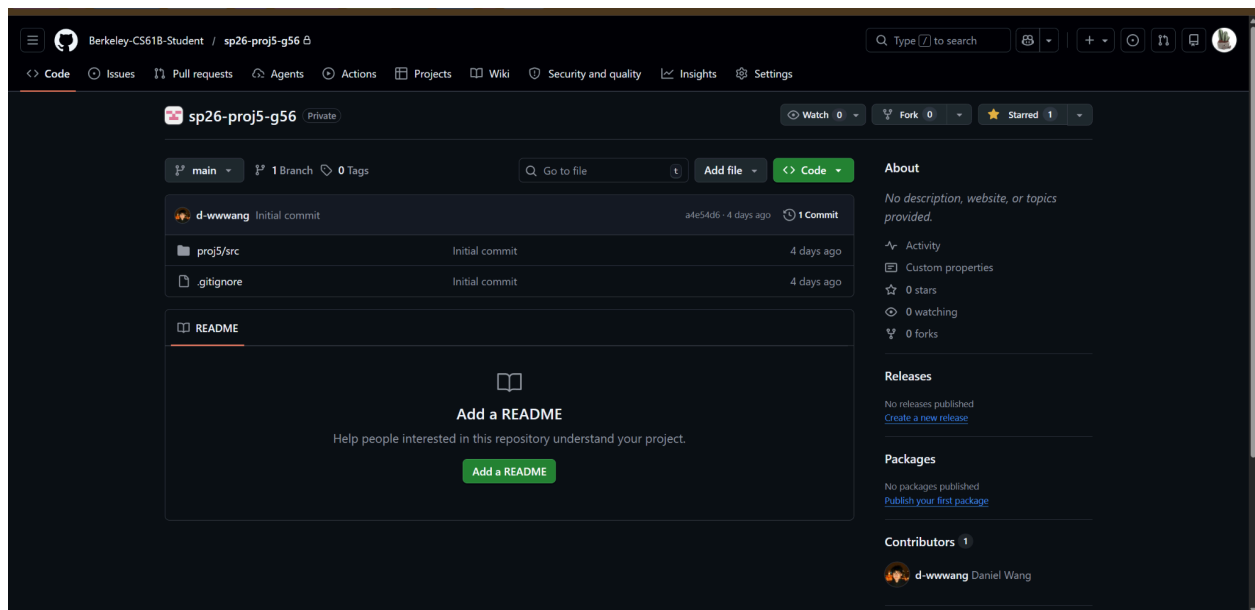
Author: Kaifeng Lu & Tyler Zeng

group:g56

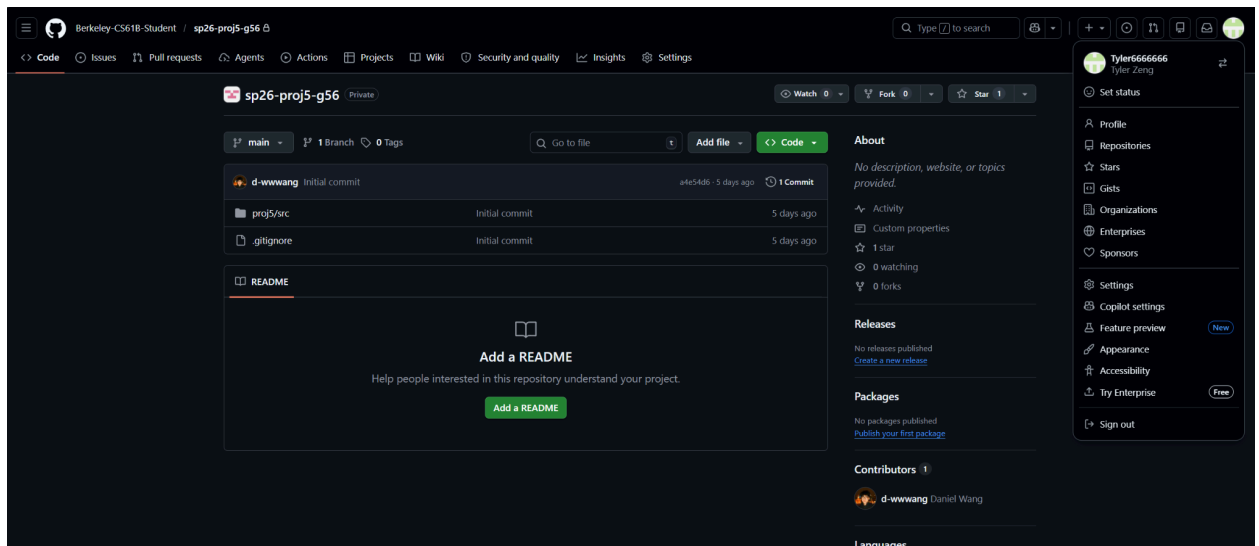
Last Updated: Apr. 15, 2026

Github Repository Screenshots

- Kaifeng Lu:



- Tyler Zeng:



Data

- Save.txt:

We plan to store the following progress of game as comma separated values, which can be used to reconstruct the current status of the game:

- Seed of world, int
- Current (X,Y) coordinate for the user on the map. (int,int) 2-tuple
- Current (X,Y) coordinate for the monster on the map. (int,int) 2-tuple
- A list of (X,Y) coordinates for remaining coins on the map. (int,int) 2-tuples
- Current score, int.

Data Structures

What data structures will you be using? How will your data structures use the data? How will your program use the data structures?

Variable	Data Structure	Purpose	Implementation
Room	Customized Node	Store the information of a Room, which is also the vertex of the graph for minimum spanning tree. - (X,Y) coordinate of the center - Width and height of the room	Hashable Customized Node
Hallway	Customized Node	Store the information of a Hallway, which is also the undirected edge for minimum spanning tree: - From Node - To Node - Length of the Hallway (weight) - (X,Y) coordinate of the gate in From - (X,Y) coordinate of the gate in To	Hashable Customized Node
Layouts	Adjacent List	The rooms-hallways form a graph of minimum spanning tree, where vertices are rooms and edges are hallways.	HashMap<Node, Node>
rooms	Set	Keep track of all rooms created	HashSet
hallways	Set	Keep track of all hallways created	HashSet
floors	Set	Keep track of all floors	HashSet
world	2-D array	Keep track of all tiles in the world	2-D array of Tiles
Bogeyman	Customized Tile	Keep track of motion and behavior of Bogeyman chasing for the player	Extends Tile

User	Customized Tile	Keep track of the motion, HP, score, and behavior of player	Extends Tile
Echo Buffer	2-D array	Stores recently detected tiles and fades them out over time	2-D array of Nodes

Algorithms

World Generation

1. Fix the width and height of the given world, then initialize the TEREnderer
2. Initialize all tiles by nothing
3. Generate and place random rooms of different sizes on the map:
 - a. **Randomly** generate a (x,y) tuple for the center of a new room. Must be at least 5 tiles away from the frame.
 - b. **Randomly** generate a (w,h) tuple for the width and height of the new room, bounded in range [4,12].
 - c. Check if the new room **overlaps** with any existing room.
 - d. If not, draw the room world, and add it as a node of the undirected graph. Otherwise, start again with 3a.
 - e. Repeat 3a-3d until the total area of the room exceeds 80% of the canvas.
4. Construct a minimum spanning tree on the graph as the hallways that connects them:
 - a. Find the **pairwise shortest hallways** that connect the **boundaries** of two rooms, then add those hallways as the edge to the graph. (How?)
 - b. Perform a **minimum spanning tree** algorithm to find the pairwise shortest hallways.
 - c. **Reduce the graph** to the minimum spanning tree
 - d. Draw the remaining hallways on the world matrix.
5. Spawn the user, objectives and enemy, if setup, in the graph
 - a. The user, objective, and enemy must be in **different rooms**.

Main Menu

- Small menu state variable
- Checks for valid key input for random seeds

Interactivity

- Navigation

For navigation and enemy behavior, we can use A* pathfinding so monsters and players can move intelligently through connected rooms and corridors.

- Update the movements
 - Input handling
 - Collision detection: checks whether the next tile is walkable.

- Update the player's position
- Ray tracing
 - Rays are traced across the map using **tile stepping**
 - Each ray checks tiles one by one until it reaches its maximum range, hits a wall, or loses too much intensity
 - Each ray checks tiles one by one until it reaches its maximum range, hits a wall, or loses too much intensity
- Interaction with entities
 - player steps onto a coin tile, the coin is removed and the score is increased
 - player reaches the exit, the game ends.
 - player takes damage and HP reaches zero after the step, the game ends immediately.
- Render
 - uses the **echo buffer** to determine what should currently be visible.
 - Tiles with stronger recent echoes are drawn more clearly, while older or weaker echoes are drawn dimly or not at all.
 - Tiles to consider:
 - temporary echo visualization of walls and corridors,
 - any detected coins or objects,
 - the player,
 - and the Bogeyman's red rays if they are active
 - fading effects based on stored intensity values
 - This is repeated every frame

HUD

- HP: user starts with 5 Hearts
- Score: the current score of user
- Time: the game time

Saving & Loading

- We store everything in the .txt file to reconstruct the world from beginning:
 - Seed of the game to reconstruct layout
 - User
 - Coordinates
 - Score
 - HP
 - Bogeyman
 - Coordinates
 - Status Chasing
 - Remaining coins
 - Coordinates

Ambition Features

- Primary feature: Blind Search with Echoes
 - Lost in total darkness, the player **cannot see walls, paths, or the exit**. Instead, every step **sends out sharp echo** pulses that bounce through the space and briefly **reveal the shape** of nearby rooms, corridors, and hidden dangers. By reading these fading sound reflections, the player must navigate the unknown and find the way out.
 - For the **echolocation system**, the main algorithm is **2D raycasting**. Every time the player moves, 8 sound rays are emitted in the compass directions and traced through the tile map. These rays detect walls, monsters, and objectives, then optionally reflect when they hit surfaces. Their intensity fades over distance, which creates the temporary “mental image” of nearby space. On a grid, this can be implemented efficiently with **tile stepping**, **Bresenham line checks**, or **DDA ray traversal(to be determined)**.
 - We also maintain a short-lived **memory map** or **echo buffer**, which stores recently detected tiles and fades them out over time. This gives the player a readable but incomplete sense of space, preserving the tension of blindness while still making exploration playable.

Complexity

- World generation:
 - A simple room placement system typically runs in about $O(N^2)$
 - Corridor construction between rooms is usually $O(N)$ or $O(N \log N)$
 - The full map itself requires $O(W \times H)$ space
- Echos
 - Each step taken by the player emits 8 rays, This gives a runtime of roughly $O(8L)$ per movement.
 - Updating the echo buffer also depends on how many tiles are touched by these rays, so it remains proportional to ray travel distance
- Navigation:
 - single A* search typically takes: $O(N \cdot \log N)$ time and $O(N)$ space in the worst case
- Other game systems are typically $O(1)$

Question

Open Questions

1. How long should the echo buffer remain visible before it fades completely?
2. Should the player be able to emit a stronger manual pulse without moving, or should pulses only happen on movement?
3. How many rebounds should a sound ray be allowed before it disappears?

4. Should the exit be visible through echolocation from the start, or only after the main task is completed?
5. How dangerous should the Bogeyman be? Should contact always damage the player, or only under certain conditions?
6. Should monsters actively respond to the player's sound pulses, making echolocation both useful and risky?

Closed Questions

1. The game uses a 2D tile-based map made of rooms connected by corridors.
2. The player cannot directly see the environment and must navigate using echolocation.
3. Each player movement emits 8 sound rays in the compass directions.
4. Sound rays fade with distance and also fade over the next few steps after being emitted.
5. The game uses an echo buffer to store recent sound information as a temporary mental image.
6. The Bogeyman emits red rays when it moves, allowing the player to detect its position.
7. The player has an HP system, and the game ends if HP reaches zero after a step.
8. The map contains coins, which contribute to the score and encourage exploration.
9. The level includes an objective/task and an exit, giving the player a clear progression goal.
10. The gameplay loop combines navigation, survival, collection, and escape.

Diagram

Draw a diagram summarizing your design. The diagram should capture the most important points about your design.

Example

If you need a guide on what to draw for the diagram, explain what happens during world generation and/or the core game loop. Show how user inputs affect the state of the world, and how that new world state is displayed to users. Draw out the flow of methods called and the data structures that are passed around.

